CHAUDHARY CHARAN SINGH UNIVERSITY (CCSU) • BCA V SEMESTER

BCA-502: Java Programming & Dynamic Webpage Design

Session: December 2025 Examination | **Paper Code:** 18022 | **Authentic University Solved Paper (All Questions Solved)** | **Time:** 3 Hours • **Max Marks:** 75

SECTION A — VERY SHORT ANSWER QUESTIONS (5 × 3 = 15 Marks)

Q1 (3M) What is the difference between Process-based and Thread-based multitasking in Java?

Comparison:

Feature	Process-Based Multitasking	Thread-Based Multitasking
Granularity	Heavyweight; each process has its own separate address space.	Lightweight; threads share the same address space in a single process.
Context Switching	High overhead; managed by operating system kernel.	Low overhead; switches quickly within the JVM.
Communication	Requires Inter-Process Communication (IPC, sockets, pipes).	Direct shared memory, wait(), notify(), synchronized blocks.

Q2 (3M) Explain the 'super' keyword in Java with usages.

Usage of 'super': A reference variable used in a subclass to refer directly to its immediate parent class object:

1. **Invoke Parent Constructor:** `super(args);` — must be the first line in subclass constructor.
2. **Access Parent Variables:** `super.variableName` — accesses shadowed parent instance fields.
3. **Call Parent Methods:** `super.methodName()` — invokes overridden parent class methods.

Q3 (3M) What is Bytecode in Java? Why does it enable platform independence?

Explanation: Java source code (`.java`) is compiled by `javac` into an intermediate machine-independent format called **Bytecode** (stored in `.class` files). Any platform equipped with a standard Java Virtual Machine (JVM) interprets and executes this bytecode natively, achieving "Write Once, Run Anywhere" (WORA).

Q4 (3M) Explain Garbage Collection in Java.

Definition: Java features automatic memory management via the Garbage Collector (GC), a background daemon thread that periodically scans the Heap memory and deallocates memory occupied by unreachable (unreferenced) objects, preventing memory leaks without manual pointer freeing.

Q5 (3M) What is the purpose of the 'finally' block in Exception Handling?

Definition: The `finally` block always executes regardless of whether an exception is thrown, caught, or unhandled. Used to perform critical resource clean-up actions such as closing file streams, database connections, and network sockets.

SECTION B — SHORT ANSWER QUESTIONS (2 × 7.5 = 15 Marks)

Q6 (7.5M) **Differentiate between Abstract Class and Interface in Java with examples.**

Comparison:

Aspect	Abstract Class	Interface
Methods	Can have abstract and concrete methods with state.	Contains abstract methods (and default/static in Java 8+).
Variables	Can have instance variables (any access modifier).	All fields are implicitly `public static final` constants.
Inheritance	Single inheritance (a class `extends` only 1 abstract class).	Multiple inheritance (a class can `implements` multiple interfaces).

Q7 (7.5M) **Explain Exception Handling mechanism in Java with keywords: try, catch, finally, throw, and throws.**

Keywords & Mechanism:

- `try` : Encloses code that may potentially throw an exception.
- `catch` : Catches and handles specific exception types.
- `finally` : Guaranteed cleanup block executed after try/catch.
- `throw` : Explicitly throws an exception object (e.g., `throw new ArithmeticException("Divide by zero");`).
- `throws` : Declares exceptions in method signatures for callers to handle.

Q8 (7.5M) **Explain the Applet Life Cycle and its core callback methods.**

Lifecycle Methods:

1. `init()` : Called once when the applet is first loaded into the browser/appletviewer to initialize variables.
2. `start()` : Invoked every time the applet becomes visible or browser tab is opened.
3. `paint(Graphics g)` : Renders shapes, text, and images on the screen surface.
4. `stop()` : Invoked when the user leaves the page or minimizes the browser.
5. `destroy()` : Called before the applet is completely reclaimed from memory.

SECTION C — DETAILED ANSWER QUESTIONS (3 × 15 = 45 Marks)

Q9 (15M) **Write an HTML code to create a User Registration Web Form containing: Username, Date of Birth, Address, Gender (Radio buttons), Email-ID, Mobile Number, and Submit/Reset buttons.**

HTML Code:

```
<!DOCTYPE html>
<html>
<head>
    <title>User Registration Form</title>
</head>
<body>
    <h2>Student / User Registration Form</h2>
    <form action="register" method="POST">
        <table cellpadding="6">
            <tr>
                <td>User Name:</td>
                <td><input type="text" name="username" required /></td>
            </tr>
            <tr>
                <td>Date of Birth:</td>
                <td><input type="date" name="dob" required /></td>
            </tr>
            <tr>
                <td>Address:</td>
                <td><textarea name="address" rows="3" cols="30"></textarea></td>
            </tr>
            <tr>
                <td>Gender:</td>
                <td>
                    <input type="radio" name="gender" value="Male" /> Male
                    <input type="radio" name="gender" value="Female" /> Female
                </td>
            </tr>
            <tr>
                <td>Email-ID:</td>
                <td><input type="email" name="email" required /></td>
            </tr>
            <tr>
                <td>Mobile Number:</td>
                <td><input type="tel" name="mobile" pattern="[0-9]{10}" required /></td>
            </tr>
            <tr>
                <td colspan="2" align="center">
                    <input type="submit" value="Register Now" />
                    <input type="reset" value="Clear Form" />
                </td>
            </tr>
        </table>
    </form>
</body>
</html>
```

Q10 (15M) **What is JDBC? Explain the Four Types of JDBC Drivers with architectural diagrams and characteristics.**

Definition: Java Database Connectivity (JDBC) is an industry-standard Java API that enables Java programs to interact with relational databases using SQL queries.

Four Types of JDBC Drivers:

1. **Type-1: JDBC-ODBC Bridge Driver:** Translates JDBC calls to ODBC calls. Requires ODBC driver installation on every client machine. Obsolete and slow.
2. **Type-2: Native-API / Partially Java Driver:** Converts JDBC calls directly into native C/C++ vendor database client libraries (e.g., Oracle OCI). Requires client-side installation.
3. **Type-3: Network Protocol / All-Java Middleware Driver:** Translates JDBC calls into an intermediate server middleware protocol, which then talks to multiple databases. Flexible for 3-tier architectures.
4. **Type-4: Thin Driver / Direct-to-Database Pure Java Driver:** Converts JDBC calls directly into vendor-specific database network protocols via TCP/IP sockets. 100% pure Java, zero client installation, highest performance (Modern standard!).

Q11 (15M) **Explain Servlet Life Cycle and compare it with JSP Life Cycle. Write a complete HttpServlet code example.**

Servlet Life Cycle Methods:

1. `init(ServletConfig config)` : Loads servlet class, creates instance, and initializes parameters (called once).
2. `service(HttpServletRequest req, HttpServletResponse res)` : Dispatches to `doGet()` or `doPost()` for every incoming HTTP request.
3. `destroy()` : Invoked when container unloads the servlet, releasing database and memory resources.

Servlet Implementation Example:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class WelcomeServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String user = request.getParameter("username");
        out.println("<html><body>");
        out.println("<h2>Welcome, " + user + "!</h2>");
        out.println("<p>Server Response Time: " + new java.util.Date() + "</p>");
        out.println("</body></html>");
    }
}
```

Q12 (15M) **Define Thread in Java. Explain how to create threads using (a) Thread class and (b) Runnable interface. Write a complete multithreaded program.**

Multithreading in Java:

```
// Approach 1: Extending Thread class
class NumberPrinter extends Thread {
    public void run() {
        for(int i = 1; i ⩽ 5; i++) {
            System.out.println("Thread 1: " + i);
            try { Thread.sleep(500); } catch(Exception e) {}
        }
    }
}

// Approach 2: Implementing Runnable interface
class LetterPrinter implements Runnable {
    public void run() {
        for(char c = 'A'; c ⩽ 'E'; c++) {
            System.out.println("Thread 2: " + c);
            try { Thread.sleep(500); } catch(Exception e) {}
        }
    }
}

public class MultiThreadDemo {
    public static void main(String[] args) {
        NumberPrinter t1 = new NumberPrinter();
        Thread t2 = new Thread(new LetterPrinter());
        t1.start(); // Spawns new thread
        t2.start();
    }
}
```

Q13 (15M) **What are Layout Managers in AWT/Swing? Explain BorderLayout, FlowLayout, and GridLayout with examples.**

Layout Managers: Automatically position, size, and rearrange GUI components inside containers regardless of screen resolution or OS window scaling.

- **FlowLayout:** Arranges components sequentially from left to right, wrapping to the next line when space runs out. Default for `Panel` and `Applet`.
- **BorderLayout:** Divides container into 5 regions: `NORTH`, `SOUTH`, `EAST`, `WEST`, and `CENTER`. Default for `Frame` and `JFrame`.
- **GridLayout:** Arranges components into a rectangular grid of equal-sized rows and columns.
